

SIMATS ENGINEERING

CO3-ASSESSMENT TOOL 1

Experiments

COURSE NAME: Computer Networks

COURSE CODE: CSA0706

COURSE OUTCOME COVERED

CO3: *Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions*

Assessment Tool Weightage: 40%

Dr.V.Parthipan

Code of Conduct:

I **M Nishanth (192511154)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M Nishanth

Criteria	Understanding of Concepts 2.5	Experimental Design 3	Use of Tools 2	Analysis of Results 1.5	Documentation 1	Tot al (10)
Q1						9
Q2						9
Q3						9
Q4						9

Faculty Signature

Experiments

QUESTIONS:4

Total Marks:40

1. Capture packets during a TCP connection setup. Identify the three packets involved in the handshake (SYN, SYN-ACK, and ACK). Demonstrate the role of each flag and how they establish a reliable connection. (BL4) (10)
2. Capture a DNS query using UDP in Wireshark. Compare the UDP header size with TCP and note the absence of sequence numbers or acknowledgments. Discuss how this lightweight design impacts speed and reliability. (BL4) (10)
3. Simulate packet loss (e.g., disconnect briefly) and capture traffic in Wireshark. Observe how TCP retransmits lost packets while UDP does not. Explain why this difference makes TCP reliable but slower compared to UDP. (BL4) (10)
4. Use Wireshark to capture DNS traffic while resolving a domain name. Identify the query type (A, AAAA, MX) and the response received. Measure the time taken for resolution and explain why DNS typically uses UDP. (BL4) (10)

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

SOLUTIONS:

The image shows a Wireshark packet capture analysis. The packet list on the left shows a packet of type **Transmission Control** (TCP) with a sequence number of 247740104. The packet details pane on the right shows the following information:

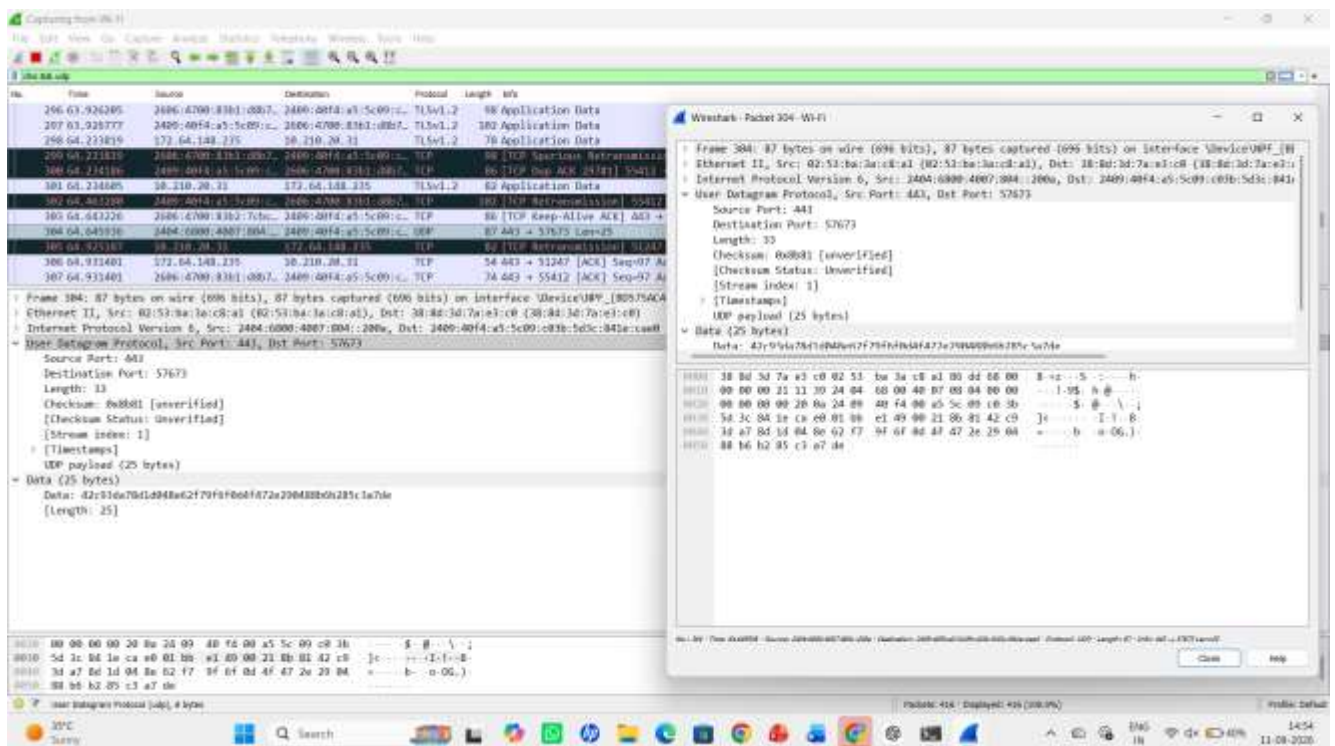
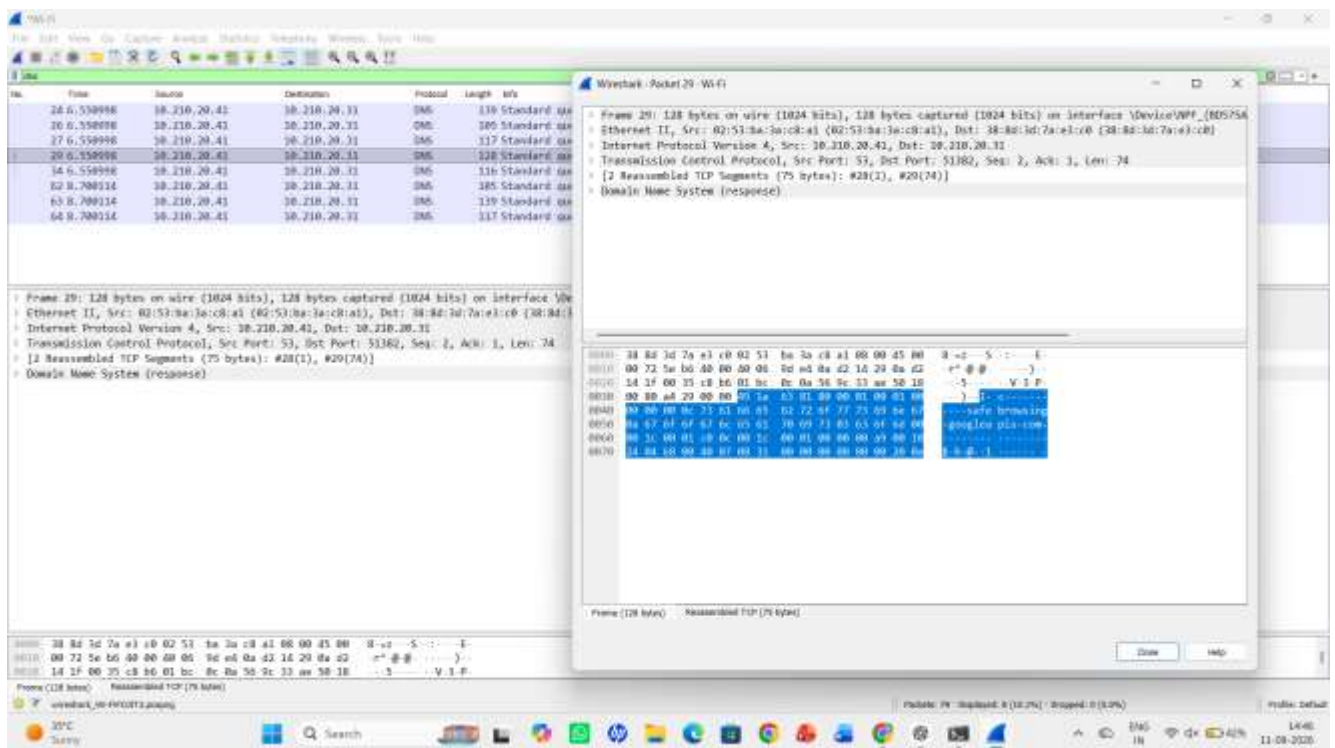
- Source Port:** 62853
- Destination Port:** 53
- Stream Index:** 20
- Conversation completeness:** Complete, WITH_DATA (31)
- TCP Segment Len:** 0
- Sequence Number:** 0 (relative sequence number)
- Sequence Number (raw):** 247740104
- Next Sequence Number:** 1 (relative sequence number)
- Acknowledgment Number:** 0
- Acknowledgment number (raw):** 0
- Header Length:** 32 bytes (8)
- Flags:** **RST (5)**
- Window:** 65535
- Checksum:** 8640c7
- Checksum Status:** Unverified
- Urgent Pointer:** 0
- Options:** (12 bytes)
- Timestamps:**

The packet bytes pane at the bottom shows the raw data of the packet, including the TCP header and the RST flag.

The image shows a Wireshark packet capture analysis. The packet list on the left shows a packet of type **Transmission Control** (TCP) with a sequence number of 247740104. The packet details pane on the right shows the following information:

- Source Port:** 62853
- Destination Port:** 53
- Stream Index:** 20
- Conversation completeness:** Complete, WITH_DATA (31)
- TCP Segment Len:** 0
- Sequence Number:** 0 (relative sequence number)
- Sequence Number (raw):** 247740104
- Next Sequence Number:** 1 (relative sequence number)
- Acknowledgment Number:** 0
- Acknowledgment number (raw):** 0
- Header Length:** 32 bytes (8)
- Flags:** **RST (5)**
- Window:** 65535
- Checksum:** 8640c7
- Checksum Status:** Unverified
- Urgent Pointer:** 0
- Options:** (12 bytes), Maximum segment size, No-Operation (NOP), Window scale, No-Operation (NOP), No-Operation (NOP), SACK permitted
- Timestamps:**

The packet bytes pane at the bottom shows the raw data of the packet, including the TCP header and the RST flag.



Wireshark interface showing a packet capture of a TCP segment. The packet list shows a packet from 10.0.0.1 to 10.0.0.2, port 80. The packet details pane shows the following information:

- Ethernet II, Src: 10:00:00:00:00:00 (10:00:00:00:00:00), Dst: 10:00:00:00:00:00 (10:00:00:00:00:00)
- Internet Protocol Version 4, Src: 10.0.0.1, Dst: 10.0.0.2
- Transmission Control Protocol, Src Port: 80, Dst Port: 80, Seq: 1, Len: 0
- Source Port: 80
- Destination Port: 80
- [Stream index: 0]
- [Conversation completeness: Incomplete, DATA (15)]
- [TCP Segment Len: 0]
- Sequence Number: 1 (relative sequence number)
- Sequence Number (raw): 105733572
- [Next Sequence Number: 1 (relative sequence number)]
- Acknowledgment Number: 0
- Acknowledgment Number (raw): 0
- 1000 Header Length: 32 bytes (8)
- Flags: 0x00 (0)
- Window: 0
- [Calculated window size: 0]
- Checksum: 0x0000 (unverified)
- [Checksum Status: Unverified]
- Urgent Pointer: 0
- Options: (12 bytes), Maximum segment size, No-Operation (NOP), Window scale, No-Operation (NOP), No-Operation (NOP), SACK permitted
- [Timestamps]

The packet bytes pane shows the raw data of the packet:

```
0000 02 53 5a 3a c8 a1 30 b1 3d 7a e3 c0 00 00 00 00  5:  S: 80 -> 80
0010 ad 49 00 20 00 fe 24 09 40 f4 00 a3 5c 09 c0 35  1:  S: 80 -> 80
0020 5d 3c 84 1e ca e0 25 03 10 63 00 27 00 01 00 00  3:  S: 80 -> 80
0030 00 00 00 00 00 14 f1 30 01 5b 5c 65 00 00 00 00  0:  S: 80 -> 80
```

The packet bytes pane also shows the packet structure:

```
0000 02 53 5a 3a c8 a1 30 b1 3d 7a e3 c0 00 00 00 00  5:  S: 80 -> 80
0010 ad 49 00 20 00 fe 24 09 40 f4 00 a3 5c 09 c0 35  1:  S: 80 -> 80
0020 5d 3c 84 1e ca e0 25 03 10 63 00 27 00 01 00 00  3:  S: 80 -> 80
0030 00 00 00 00 00 14 f1 30 01 5b 5c 65 00 00 00 00  0:  S: 80 -> 80
```